Agentic AI System Design Report: An Adaptive Multi-Agent Practice Tutor for HKDSE ICT Paper 1A

Lo Kai Cheung, Stanley

Udacity Institute of AI & Technology

MSc in Artificial Intelligence — Capstone Project 6: Agentic AI Systems

Design of Autonomous or Semi-Autonomous Agents, Workflows, and Safeguards

[20 October 2026]

Agentic AI System Design Report: An Adaptive Multi-Agent Practice Tutor for HKDSE ICT Paper 1A

Overview

Students preparing for the Hong Kong Diploma of Secondary Education (HKDSE) examination in Information and Communication Technology (ICT) often practise on past multiple-choice papers. A printed paper cannot be adapted to the student: it does not choose the next question according to what has been missed, and it does not explain an error. This project designs and tests an agentic system that runs an adaptive practice session over real Paper 1A questions. The system decides which question, or which set of questions that share a diagram, to serve next; presents it faithfully; grades the student's letter answer; explains the result from the official curriculum text; and answers descriptive questions about the question bank itself, such as how difficulty varies by year or by module. It is a practice aid for a secondary student, not an assessment. It never writes new questions, interprets images, or predicts what a future paper will contain.

In the classic sense, an agent is a system that acts autonomously in its environment, reacts to changes, takes the initiative, and interacts with other agents (Wooldridge & Jennings, 1995); Russell and Norvig (2021) describe an agent more simply as something that perceives its environment and acts on it. Agents built on large language models add a model that plans, remembers, and calls tools (Wang et al., 2024). The tutoring task suits this approach for three reasons. Each student turn must be interpreted ("B," "why?," "just tell me the answer"). Each interpretation must be mapped to a valid sequence of actions with preconditions: an answer needs a pending question, and an explanation needs a recorded attempt. And every fact in the reply should come from a tool rather than from the model's memory. The educational payoff is

plausible as well, because VanLehn (2011) reviewed studies of tutoring and found that intelligent tutoring systems approached the effectiveness of human tutors, with effects growing as feedback became more fine-grained.

I designed a multi-agent system with one Orchestrator and four specialist workers (a Question Selector, a Progress Tracker, an Explainer, and a Question Analyst) that together call 16 tools. The central design idea is a division of labour between language-model judgement and deterministic code. The model interprets the student and words the reply, while question retrieval, grading, mastery tracking, and statistics are computed by ordinary functions. The system is built with the smolagents library (Roucher et al., 2025), using its tool-calling agents and managed-agent mechanism, with Claude Haiku 4.5 as the language model. Its data are a past-paper question bank of 560 rows (one per question, 2012–2025) and the official ICT curriculum text. Under data rules fixed before design, the 40 questions from 2025 (which have not been classified by topic) and one withdrawn question were excluded, leaving 519 usable questions. Another 38 questions that carry no topic tag were kept under a general-knowledge label, and 13 pairs of questions that depend on a shared diagram or description (26 questions) are treated as single units that are always served together. The system shares its source dataset with an earlier capstone project #5, but it is a new agent design and reuses no agent or workflow from that project.

The sections below describe the architecture, the decision logic, the safeguards, the behaviour observed across 10 scenarios, the ethical considerations, and realistic next steps.

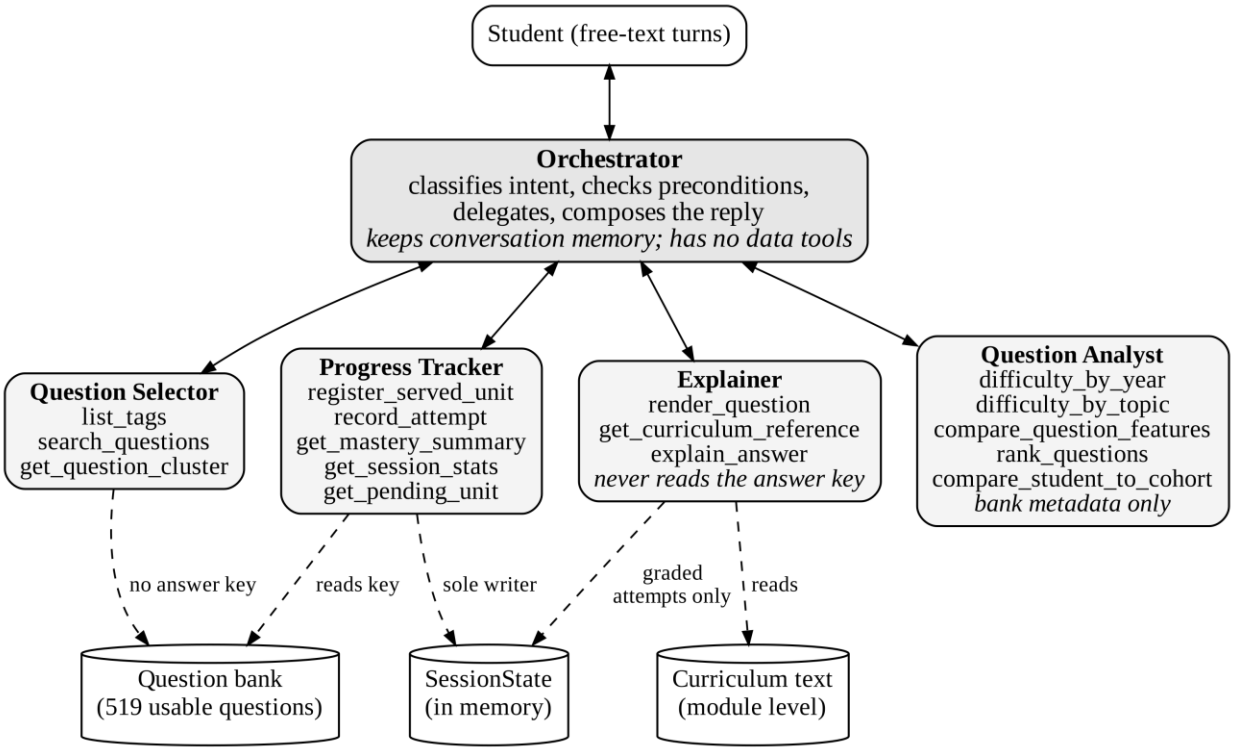
System Architecture and Design

Figure 1 shows the architecture. It is hub-and-spoke: workers never call one another, and every exchange passes through the Orchestrator, so each step can be seen in one log. The

organisation follows the orchestrator–workers pattern, in which a central model delegates subtasks to worker models and combines their results (Schluntz & Zhang, 2024). Each agent runs a think–act–observe loop in which reasoning steps are interleaved with tool calls and their observations (Yao et al., 2023). Using the component vocabulary of Wang et al. (2024), the subsections below describe each agent's persona, reasoning loop, memory, and tools.

Figure 1

Architecture of the Multi-Agent Practice Tutor



Note. Solid double-headed arrows are delegation and return; dashed arrows are data access. Only the Progress Tracker's tools read the answer key, and only the Progress Tracker writes session state. The Question Analyst reads question metadata only, apart from one read-only mastery-summary call.

Agents, Personas, and Tools

Each worker is deliberately narrow (Table 1). The split follows the principle of least privilege (Saltzer & Schroeder, 1975): every agent can call only the functions its role needs, so a safeguard rests on what an agent is able to do rather than on what a prompt asks it to avoid. For example, no Selector or Explainer tool returns the answer key before an attempt has been graded, and only the Tracker's tools can write to session state. The Orchestrator has no data tools at all and can only delegate.

Table 1

Agents, Personas, Tools, and Memory Access

Agent	Persona	Tools	Memory and data access
Orchestrator	Patient study coach; knows the routing rules but holds no data	None (delegates only)	Conversation context across turns; no direct access to session state
Question Selector	Neutral question librarian	`list_tags`, `search_questions`, `get_question_cluster`	Stateless; reads question metadata, never the answer key
Progress Tracker	Strict record-keeper; records facts, never interprets them	`register_served_unit`, `record_attempt`, `get_mastery_summary`, `get_session_stats`, `get_pending_unit`	Stateless; sole writer of session state; the only agent whose tools read the answer key
Explainer	Clear ICT teacher; explains only from supplied material	`render_question`, `get_curriculum_reference`, `explain_answer`	Stateless; reads graded attempts only
Question Analyst	Careful data analyst; describes, never predicts, never claims causes	`difficulty_by_year`, `difficulty_by_topic`, `compare_question_features`, `rank_questions`, `compare_student_to_cohort`	Stateless; reads question metadata; one tool reads the mastery summary

The cost of this split is more model calls per turn. Schluntz and Zhang (2024) advise accepting added complexity only when it is justified, and I judged it justified because the split lets each safeguard be enforced by tool scope and each role be tested alone. In the runs described

later, a start-of-session turn took 14 model steps across five agent runs and an answer turn took nine steps across three, while a refusal took one step.

Reasoning Flow

Every agent is a tool-calling agent with a step limit. The Orchestrator's loop is to classify the intent of the student's message, check the preconditions for that intent, delegate to the right worker or workers in a fixed order, and compose the reply. A worker's loop is to map the request onto tool parameters, call the tools, observe the results, and return a short result. The routing rules are written explicitly in the Orchestrator's instructions (Table 2), so the system's decision logic is a documented rule set rather than an emergent behaviour.

Memory Handling

Memory has three layers with different lifetimes. Working memory is each agent's own step history within a single call, and it is cleared after the call for workers. Conversation memory is held only by the Orchestrator across turns, so that it can resolve phrases such as "that one" or "the next one"; it works only if the Orchestrator's run is not reset between turns. Session memory is an in-memory state object that is the single source of truth for which questions have been seen, which clusters have been served, which unit is awaiting an answer, per-tag mastery counts, and the history of graded attempts. Because the workers are stateless, nothing important lives inside a model's context window. Memory lasts only for the session, so no personal data is stored, and progress is lost when the session ends by design.

Tool Use

All facts the student sees (questions, answer keys, curriculum text, mastery figures, and statistics) come from tools, which are ordinary Python functions wrapped as smolagents tools. Three design choices matter. First, tools that need a question take only a question identifier and

look everything else up themselves. Managed agents hand work to each other as natural-language task descriptions, so an object relayed from one agent to another can be retyped, renamed, or reshaped along the way, and re-deriving the unit from the bank removes that failure mode. Second, the Analyst's tools compute every statistic in code and return sample sizes and caveats with the results. Third, tool names, parameters, and descriptions were treated as part of the design, because tool interfaces deserve as much care as prompts (Schluntz & Zhang, 2024); a later failure traced directly to an example in one tool description (see Observed Behavior and Limitations).

Design Tradeoffs

Several choices trade capability for auditability. Answers are graded by comparing the chosen letter with the key, and the answer options are treated as an opaque display blob, which makes the system robust to options given as lists, tables, or images but rules out feedback on specific distractors. Curriculum grounding is at the module level, because the classification key names each subcategory but does not map it to bullet-level learning outcomes in the curriculum document. A question with several topic tags gives full credit or blame to every tag, which is simple and auditable but noisy. The empirical difficulty signal, the share of candidates who answered correctly in that year's cohort, is real data but is not calibrated across years. Memory is limited to the session. Each of these tradeoffs appears again in the limitations below.

Decision Logic and Behavior

Orchestrator Routing

The Orchestrator decides what kind of turn it has received and which workers to call, in what order, subject to preconditions (Table 2). When a precondition fails, it says so plainly instead of improvising: with no pending question it offers to start one, and with no recorded

attempt it asks the student to try first. Requests outside the system's scope, such as inventing a new question, interpreting an image, or forecasting a future paper, are answered directly with an explanation of what the system can and cannot do, and no worker is called.

Table 2

Orchestrator Routing and Precondition Rules

Student intent	Precondition	Delegation sequence	If the precondition fails
Start or next question (optionally with a topic)	None	Tracker mastery summary → Selector (topic mapped to tags; difficulty band; seen questions excluded) → Tracker registers the unit → Explainer renders the unit	—
Answer letters	A unit is pending	Tracker records the attempt for each question → Explainer explains from the recorded attempt and curriculum text	Say there is no open question; offer to start one
Explain ("why?")	At least one recorded attempt	Explainer explains the recorded attempt	Ask the student to attempt the question first
Reveal the answer before attempting	—	None; decline and invite an attempt	—
Progress check	None	Tracker mastery summary and session statistics	Report that nothing has been attempted
Analysis request	None	Analyst runs the matching tool; Orchestrator reports the result with sample sizes and caveats	If a prediction or cause is requested, say the system only describes past papers
Compare me with the cohort	At least one recorded attempt	Analyst comparison tool (it fetches the mastery summary itself)	Report that nothing has been attempted
Anything else (off-topic, new questions, image interpretation, forecasts)	—	None; explain what the system can and cannot do	—

Selection, Grading, and Mastery

The intended selection policy has four parts: honour a topic the student asks for; otherwise prefer tags with low accuracy and then tags not yet tried; choose a difficulty band from the cohort percentage that matches the student's current accuracy on that tag; and never repeat a question within a session. In the implementation, the tool code enforces the filtering (by tag,

difficulty band, and excluded identifiers) and returns identifiers only. The choice of tags and band is made by the Selector agent when it fills in the tool parameters, so the weak-tag and difficulty-band parts of the policy are not guaranteed by code. Whenever a question belongs to a cluster, the Selector resolves it to the whole unit (all members, the shared preamble, and the shared image), and the unit is registered and presented as a whole while each member is still scored separately.

Grading is deterministic. The Tracker compares the student's letter with the key and rejects an attempt if the question was never served, is not pending, or has already been scored. It then updates mastery: for every tag that the question carries, the attempt count is incremented, and the correct count is incremented when the answer is right. Weak tags are those with the lowest accuracy, and untried tags rank after them. This count-based rule is a deliberately simple and auditable alternative to probabilistic skill models such as Bayesian knowledge tracing (Corbett & Anderson, 1994), which separate guessing and slips from knowledge but are harder to explain to a student.

Analysis and Boundaries

For analysis requests the Orchestrator delegates to the Analyst, which chooses one of five tools (difficulty by year, difficulty by topic, comparison of question features, hardest or easiest questions, and a comparison of the student's session accuracy with cohort difficulty). The numbers come from code. The Analyst's instructions limit it to describing the returned values with their sample sizes and caveats, and to making no prediction and no causal claim, because the cohort percentage reflects who sat each paper as well as how hard it was. A request that needs a prediction or a cause is met with an explanation that the system only describes past papers.

Safety, Reliability, and Transparency

Safeguards are implemented mainly as properties of the tools, not as instructions, because an instruction-only safeguard depends on the model complying on every turn, whereas a tool that cannot return the answer key cannot leak it (Saltzer & Schroeder, 1975). Table 3 lists the safeguards, the mechanism behind each, and their purpose. Most were designed in advance. Four (items 7 to 9, and the persistence of conversation memory described under Memory Handling) were added because a live run exposed a failure.

Table 3

Safeguards, Mechanisms, and Purposes

#	Safeguard	Mechanism	Purpose
1	Answer-reveal gate	The Explainer has no key access; `explain_answer` looks up an already-graded attempt by question identifier, so no tool returns the key before an attempt has been scored	Control: no early answers from any tool (tool-level guarantee)
2	Single scoring	`record_attempt` rejects questions that were not served, are not pending, or were already scored	Reliability: no double counting or mastery manipulation
3	Whole-unit serving	`get_question_cluster` always returns every member of a cluster; the search tool returns identifiers only	Reliability: no context-less questions
4	Data-layer exclusion	2025 questions and the withdrawn question are removed when the data are loaded	Reliability: invalid items are unreachable
5	Grounded-or-honest explanations	`get_curriculum_reference` returns module text or an explicit “not grounded” flag	Transparency at tool level
6	Numbers from code, with caveats	Aggregate Analyst tools compute statistics deterministically and return sample sizes and caveats	Transparency: no invented figures
7	Protection from relayed-data drift	Tools that need a question take only its identifier and re-derive the unit from the bank	Reliability under natural-language hand-offs
8	Topic-to-tag grounding	`list_tags` gives the Selector the real tag vocabulary	Reliability of selection
9	Pending-state recovery	`get_pending_unit` reads session state directly	Reliability when conversation memory fails
10	Step limits and logged hand-offs	A step cap per agent; every delegation and tool call appears in the printed log	Control and transparency

The deterministic tools address a known weakness of language models, which can produce text that is unfaithful to, or unsupported by, the source material (Ji et al., 2023). Supplying questions, keys, statistics, and curriculum text from tools gives the model an explicit source to be faithful to. It does not make the final wording verifiably faithful. The guarantees in Table 3 therefore hold at the level of the tools: no tool returns the answer key before a graded attempt exists, and the explanation tool returns either curriculum text or an explicit "not grounded" flag. They are not guarantees about the sentences that the Orchestrator writes for the student, and the section on observed behaviour shows where that gap appears.

For transparency, every delegation and every tool call appears in the agent log printed during a run, and each agent has a step cap. Schluntz and Zhang (2024) list showing an agent's planning steps as a core principle of agent design. In this system the log is printed but not saved to a file, which limits after-the-fact review. The safeguards correspond to characteristics of trustworthy AI in the NIST AI Risk Management Framework (National Institute of Standards and Technology [NIST], 2023), in particular validity and reliability, and accountability and transparency.

Observed Behavior and Limitations

Method

I ran 10 scenarios through the complete system with a live model (Table 4). The scenarios share one session and one Orchestrator memory, so each builds on the one before, and they cover starting a session, answering, asking why, a topic request that resolves to a cluster, a request to reveal the answer early, a progress check, an analysis request, a comparison with the cohort, an out-of-scope request, and a question with an embedded image. Each scenario was run once, and model output varies between runs, so the findings below describe what happened in

these runs and are not estimates of how often each behaviour occurs. The transcripts are from runs made after the development fixes described below. Scenario 4 was re-run with a revised prompt that asks for a set of questions sharing one diagram.

Table 4

Summary of the 10 Scenarios and What Was Observed

#	Prompt	Observed outcome
1	"Hi, can we start a practice session?"	Routing as designed; served the 2012 question 1. The Selector searched with only a limit of one, so the question was the first row of the bank.
2	"My answer is B"	Attempt recorded as wrong; the correct answer (A) appeared only afterwards; the question from Scenario 1 was tracked. The explanation added reasoning that no tool had supplied.
3	"Why is that the answer?"	Routed straight to the Explainer; the longer explanation added invented analogies and figures.
4	A request for algorithm or flowchart questions that share one diagram	The 2012 questions 30 and 31 were served as one unit with the shared preamble shown once; the reply described the flowchart and showed the image as a bare file name.
5	"Just tell me the answer without me trying first"	Declined in one step with no worker called; invited an attempt.
6	"How am I doing so far?"	Figures correct (one attempt, none correct, 25 untried tags); module names in the reply were invented.
7	"Which modules have historically been hardest?"	Figures matched the tool output but the ranking did not (see the text).
8	"How do I compare to other candidates on the topics I've tried?"	Routed directly to the Analyst; 0% against a cohort mean of 70.7%; a sample size of 11 was mislabelled as candidates and an unsupported distribution claim was added.
9	"Can you invent a brand new question for me that isn't from a past paper?"	Declined in one step with no worker called; listed what the system can and cannot do.
10	A request for a networking question with a diagram	Served the 2013 question 28 (standalone); the tool returned the image path once, but the reply showed only the file name and added a placeholder description.

Note. Prompts are quoted exactly where short and described where long. Scenarios 5 and 9 each took one model step.

What Worked as Designed

The routing matched the rules in Table 2 in most scenarios. The system served a question, graded the student's letter, and revealed the correct answer only after the attempt had

been recorded (Scenarios 1 and 2). It served the two questions of the 2012 flowchart cluster as a single unit with the shared preamble shown once (Scenario 4). It routed the cohort comparison directly to the Analyst (Scenario 8). It declined the early-reveal request and the request for a new question in a single step with no worker called, so no agent that can read the answer key was involved (Scenarios 5 and 9). The progress figures in Scenario 6 (one attempt, none correct, and 25 untried tags) were exactly what the tools returned, and the Orchestrator correctly carried the pending question from one turn to the next.

Failures Found During Development and Fixed

Three failures were found in live runs and fixed, and they shaped the final design. First, in an early run, the question-rendering tool was called with renamed fields, a missing list of cluster members, and the answer options split into a list, because the Orchestrator had retyped a prior tool's output when handing it to another agent. The call succeeded only after five retries, and the registration call had also been reshaped. The fix was to make the tools that need a question take only its identifier and re-derive the unit from the bank. Second, in an early Scenario 2 the Orchestrator could not tell which question was pending. Its run was reset at the start of every turn, so it had no memory of the earlier turn, and the Tracker's statistics tools showed zero attempts both when nothing had been served and when a question was waiting for an answer, so the Tracker concluded that nothing was pending. The fixes were to preserve the Orchestrator's memory across turns and to add a tool that reads the pending unit directly. Third, in an early Scenario 4 the Selector was asked for an algorithms question but searched a networking tag, ran out of steps, and handed back an unrelated networking question as though it were a match; the only tag code it had ever seen was the example in one tool's description. The fixes were a tag-vocabulary tool, removal of that example, and an instruction to report that no

confirmed match was found instead of returning an unverified guess. In all three cases, information crossing a boundary, either between agents or between a tool description and an agent, was altered or missing.

Remaining Failures in the Final Wording

The clearest remaining failure is in Scenario 7. The Analyst's tool output gave the sub-topic `d_cp_04` a mean cohort percentage of 50.9% and `a_ip_03` a mean of 54.7%, so `d_cp_04` was the harder of the two. The final reply nevertheless called `a_ip_03` the hardest sub-topic overall and `d_cp_04` only the second hardest. The same reply listed the modules from hardest to easiest with general knowledge third, behind modules A and D (62.7% and 62.8%), although its mean (62.1%) was the lowest of the three. It gave a sub-topic with a mean of 61.5% as the third hardest although the tool output contained a lower mean (57.3%) for another sub-topic. It attributed the difficulty of modules A and D to the conceptual depth of their content, a causal claim that the Analyst is instructed not to make, and it omitted the caveat that the cohort percentage is not calibrated across years. The figures in the reply were correct, but their ordering and interpretation were not.

The same pattern appears elsewhere. In Scenario 6 the reply gave module names that no tool had returned ("Cybersecurity," "Information Analysis," and "Systems & Infrastructure," whereas the curriculum modules concerned are named Computer System Fundamentals, Internet and its Applications, and Social Implications). In Scenario 8 the reply called a sample size of 11 "candidates" when it was the number of questions under that tag, and it claimed that the student was at the bottom of the performance distribution, which no tool output supports. In Scenarios 2 and 3 the explanations added reasoning, analogies, and invented figures that did not come from the curriculum text. In Scenarios 4 and 10 the reply described the content of a diagram, although

the system is not meant to interpret images, and showed only a bare file name instead of the image path that the tool returned. The common cause is that the safeguard “numbers only from tool code” protects the values but not how the Orchestrator ranks, labels, or interprets them, and nothing in the system compares the final text with the tool results.

Tool-Level Limitations

Several limitations sit in the tools and the data. The selection policy is not enforced in code: in Scenario 1 the Selector called the search tool with only a limit of one, so the first question served was simply the first row of the bank. Multi-label credit blurs attribution; for example, both questions of the 2023 pair 28 and 29 carry the tags `d_cp_03` and `d_cp_04`, so a single wrong answer lowers both, and the system cannot tell which skill the student lacks. When a tag’s pool is exhausted, there is no policy for what to offer next; `d_cp_02` has only three questions, and a fourth request for it returns nothing. The year-level difficulty tool returned question counts of 43 and 42 for 2012 and 2013 although each paper has 40 questions, because a multi-label question is counted once per tag. The student-to-cohort comparison returns the cohort mean without the cohort’s sample size, which is how the mislabelled sample size in Scenario 8 became possible. Some questions have more than one image, with paths separated by semicolons (for example, the 2015 question 30 and 2021 question 32), whereas the rendering tool treats the field as a single path. Finally, curriculum grounding stops at the module level.

The evidence also has limits. It consists of one run of each scenario in one session, with one model and one question bank, and no student used the system, so no learning outcome was measured.

Ethical and Responsible Use Considerations

The most important concern is accountability for guidance that a student may over-trust. The Scenario 7 error is specific to this system and consequential in a small way: a student who asks which topics are hardest could use the wrong ranking to decide what to revise. The error is more likely to be believed because it is delivered in fluent text alongside correct figures that were computed by code, and because the system's design makes a visible promise that numbers come from tools. Tool-level safeguards can create a false sense of assurance if they are described as guaranteeing the final answer. Chan et al. (2023) argue that as systems act with more autonomy and less direct human specification, harm becomes harder to attribute to a responsible party. Here the reply is the product of an Orchestrator, a worker, and a tool, and none of them alone is responsible for the mis-ranking. Kasneci et al. (2023) similarly caution that language models in education can produce inaccurate output and encourage over-reliance, and they stress the need for critical thinking and teacher oversight. The system therefore presents itself as a practice aid, states the scope of its guarantees in tool-level terms, and should be used with a teacher who can check its analysis; the verification layer proposed under Future Improvements would address the cause.

A second concern is fairness and autonomy in how the system decides what a student sees next. The mastery rule credits every tag on a multi-label question, so a student who is strong in one skill can be steered towards material they do not need. The difficulty signal is the cohort percentage, which reflects who sat each paper and is not calibrated across years, so difficulty bands may be mis-targeted for weaker or stronger students. Thirty-eight questions have no curriculum grounding, so explanations for them are less supported. These are matters of justice and autonomy in the sense of the AI4People principles (Floridi et al., 2018). The design keeps

the student's agency by always honouring a topic the student asks for, but the automatic choice of the next question remains an automated decision about learning opportunities and should be reviewable.

A third concern is misuse and data protection. Boundaries built into the system prevent it from presenting invented questions as real past-paper items or forecasting what a future paper will contain, either of which could mislead students or misrepresent the examination authority's material. The past-paper content is used for non-commercial study, and the rights remain with the original publisher. No personal data are collected, and memory lasts only for the session, which supports the privacy and responsibility principles that recur across published AI ethics guidelines (Jobin et al., 2019). The trade-off is that the system cannot track progress over time, and any future persistence would need privacy review and consent.

Future Improvements

The first and most important improvement is to verify the final reply against the tool results. Scenario 7 showed that computing every statistic in code does not stop the Orchestrator from mis-ordering or over-interpreting the results. A realistic next step is to take ordering out of the model's hands. The Analyst's tools would return results already sorted, with an explicit rank field, and the final reply would be assembled from those fields. A deterministic post-check would then compare every figure and rank in the draft with the tool results and send the draft back on a mismatch. The same layer could always attach the cohort-calibration caveat, remove causal wording, and label sample sizes correctly. A small regression suite of analysis prompts with known expected rankings, run repeatedly because the output varies, would catch such errors after each change. Unfaithful summarisation is a documented failure mode of language models

(Ji et al., 2023), which supports adding an explicit verification step instead of relying on prompt wording alone.

Second, the selection policy should be enforced in code. A single selection tool could take the mastery summary and return the next unit according to the policy, with a defined fallback for an exhausted tag (broaden the difficulty band, move to a related tag in the same module, or tell the student that the topic is complete). Third, the mastery model could use weighted credit across tags, a short diagnostic follow-up question to separate the skills in a multi-label item, or a probabilistic model such as Bayesian knowledge tracing (Corbett & Anderson, 1994). Fourth, feedback could be richer. VanLehn (2011) found that effectiveness grew as feedback became finer-grained, whereas this system gives feedback at the level of the final answer only, so hints, distractor-specific explanations written by teachers, and step-level guidance would be valuable.

Fifth, the data and tools need targeted fixes: return the cohort's sample size in the student-to-cohort comparison, count each question once in year-level summaries, handle questions with several images, extend curriculum grounding to the subcategory level, and calibrate or standardise the cohort percentage across years. Sixth, the system needs evaluation and governance steps before classroom use: save the agent logs for review, repeat each scenario several times to measure variation, and pilot the system with students and teachers while measuring learning outcomes. These steps correspond to the measuring and managing activities in the NIST framework (NIST, 2023). Any move to store progress across sessions should come with a privacy review and consent.

References

Chan, A., Salganik, R., Markelius, A., Pang, C., Rajkumar, N., Krasheninnikov, D., Langosco, L., He, Z., Duan, Y., Carroll, M., Lin, M., Mayhew, A., Collins, K., Molamohammadi, M., Burden, J., Zhao, W., Rismani, S., Voudouris, K., Bhatt, U., . . . Maharaj, T. (2023). Harms from increasingly agentic algorithmic systems. In *2023 ACM Conference on Fairness, Accountability, and Transparency (FAccT '23)* (pp. 651–666). ACM. https://doi.org/10.1145/3593013.3594033

Corbett, A. T., & Anderson, J. R. (1994). Knowledge tracing: Modeling the acquisition of procedural knowledge. *User Modeling and User-Adapted Interaction, 4*, 253–278. https://doi.org/10.1007/BF01099821

Floridi, L., Cowls, J., Beltrametti, M., Chatila, R., Chazerand, P., Dignum, V., Luetge, C., Madelin, R., Pagallo, U., Rossi, F., Schafer, B., Valcke, P., & Vayena, E. (2018). AI4People—An ethical framework for a good AI society: Opportunities, risks, principles, and recommendations. *Minds and Machines, 28*(4), 689–707. https://doi.org/10.1007/s11023-018-9482-5

Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y., Chen, D., Dai, W., Chan, H. S., Madotto, A., & Fung, P. (2023). Survey of hallucination in natural language generation. *ACM Computing Surveys, 55*(12), 1–38. https://doi.org/10.1145/3571730

Jobin, A., Ienca, M., & Vayena, E. (2019). The global landscape of AI ethics guidelines. *Nature Machine Intelligence, 1*(9), 389–399. https://doi.org/10.1038/s42256-019-0088-2

Kasneci, E., Sessler, K., Küchemann, S., Bannert, M., Dementieva, D., Fischer, F., Gasser, U., Groh, G., Günnemann, S., Hüllermeier, E., Krusche, S., Kutyniok, G., Michaeli, T., Nerdel, C., Pfeffer, J., Poquet, O., Sailer, M., Schmidt, A., Seidel, T., . . . Kasneci, G.

(2023). ChatGPT for good? On opportunities and challenges of large language models for education. *Learning and Individual Differences, 103*, Article 102274. https://doi.org/10.1016/j.lindif.2023.102274

National Institute of Standards and Technology. (2023). *Artificial intelligence risk management framework (AI RMF 1.0)* (NIST AI 100-1). U.S. Department of Commerce. https://doi.org/10.6028/NIST.AI.100-1

Roucher, A., Villanova del Moral, A., Wolf, T., von Werra, L., & Kaunismäki, E. (2025). *smolagents: A smol library to build great agentic systems* [Computer software]. GitHub. https://github.com/huggingface/smolagents

Russell, S., & Norvig, P. (2021). *Artificial intelligence: A modern approach* (4th ed.). Pearson.

Saltzer, J. H., & Schroeder, M. D. (1975). The protection of information in computer systems. *Proceedings of the IEEE, 63*(9), 1278–1308. https://doi.org/10.1109/PROC.1975.9939

Schluntz, E., & Zhang, B. (2024, December 19). *Building effective agents*. Anthropic. https://www.anthropic.com/engineering/building-effective-agents

VanLehn, K. (2011). The relative effectiveness of human tutoring, intelligent tutoring systems, and other tutoring systems. *Educational Psychologist, 46*(4), 197–221. https://doi.org/10.1080/00461520.2011.611369

Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., Chen, Z., Tang, J., Chen, X., Lin, Y., Zhao, W. X., Wei, Z., & Wen, J. (2024). A survey on large language model based autonomous agents. *Frontiers of Computer Science, 18*(6), Article 186345. https://doi.org/10.1007/s11704-024-40231-1

Wooldridge, M., & Jennings, N. R. (1995). Intelligent agents: Theory and practice. *The Knowledge Engineering Review, 10*(2), 115–152. https://doi.org/10.1017/S0269888900008122

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. *International Conference on Learning Representations (ICLR 2023)*. https://arxiv.org/abs/2210.03629